

AgentSentinel: An End-to-End and Real-Time Security Defense Framework for Computer-Use Agents

Haitao Hu
ShanghaiTech University
Shanghai, China
huht2022@shanghaitech.edu.cn

Yanpeng Zhao
Independent Researcher
Beijing, China
yannzhao.ed@gmail.com

Peng Chen
Independent Researcher
Shanghai, China
spinpx@gmail.com

Yuqi Chen*
ShanghaiTech University
Shanghai, China
chenyq@shanghaitech.edu.cn

Abstract

Large Language Models (LLMs) have been increasingly integrated into computer-use agents, which can autonomously operate tools on a user's computer to accomplish complex tasks. However, due to the inherently unstable and unpredictable nature of LLM outputs, they may issue unintended tool commands or incorrect inputs, leading to potentially harmful operations. Unlike traditional security risks stemming from insecure user prompts, tool execution results from LLM-driven decisions introduce new and unique security challenges. These vulnerabilities span across all components of a computer-use agent. To mitigate these risks, we propose AgentSentinel, an end-to-end, real-time defense framework designed to mitigate potential security threats on a user's computer. AgentSentinel intercepts all sensitive operations within agent-related services and halts execution until a comprehensive security audit is completed. Our security auditing mechanism introduces a novel inspection process that correlates the current task context with system traces generated during task execution. To thoroughly evaluate AgentSentinel, we present BadComputerUse, a benchmark consisting of 60 diverse attack scenarios across six attack categories. The benchmark demonstrates a 87% average attack success rate on four state-of-the-art LLMs. Our evaluation shows that AgentSentinel achieves an average defense success rate of 79.6%, significantly outperforming all baseline defenses.

CCS Concepts

• Security and privacy → Software and application security.

Keywords

Large Language Model, Computer-Use Agents, LLM-based Agent Security

ACM Reference Format:

Haitao Hu, Peng Chen, Yanpeng Zhao, and Yuqi Chen. 2025. AgentSentinel: An End-to-End and Real-Time Security Defense Framework for Computer-Use Agents. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25)*, October 13–17, 2025, Taipei. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3719027.3765064>

1 Introduction

Large Language Models (LLMs), such as GPT-4o [32] and Claude 3.7 Sonnet [2], represent the most advanced natural language processing technologies to date. These models accept user prompts, describing problems, requirements, or tasks, and generate reasonable and contextually appropriate responses. Due to their powerful task-processing capabilities, LLMs have been widely adopted across numerous domains and integrated into a variety of LLM-based applications. To further enhance their capabilities, tools are often integrated with LLMs, enabling them to interact with the external world—these enhanced systems are commonly referred to as LLM-based agents. Additionally, the development of the Model Context Protocol (MCP) [4] has significantly accelerated the integration of LLMs into such agents.

Among the diverse range of agents, one of particular significance has emerged: the computer-use agent. These agents allow LLMs to take full control of a user's computer environment and perform complex tasks much like a human user. For instance, Anthropic first introduced computer-use capabilities in Claude 3.5 Sonnet [3]. OpenAI has since followed with its Operator framework and the Computer-Using Agent model [33]. Similarly, Manus [27] is a general-purpose AI agent that can interact with a cloud-based computer environment. As a result, the ability to perform real-world computer interactions—referred to as computer-use has become a critical capability for next-generation LLM-based agents.

The use of computer-use agents inevitably introduces new vulnerabilities and significantly expands the attack surface due to their ability to interact with the computer environment. Specifically, in scenarios without an adversary, the unpredictable nature of LLMs can result in error-prone or overly complex behavior. If an LLM gains control of a user's computer and executes any operation it deems appropriate, this raises immediate and serious concerns. As a result, incorrect tool use may compromise the integrity of the user's operating system and, in severe cases, cause irreversible damage.

*Corresponding author



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

CCS '25, Taipei

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1525-9/2025/10

<https://doi.org/10.1145/3719027.3765064>

Therefore, it is critical to intercept such risky actions promptly to prevent further harm.

In adversarial settings, both the agent infrastructure and the underlying operating system are susceptible to attacks targeting different components of the computer-use agent. To compromise the agent infrastructure, an attacker could issue seemingly benign instructions designed to exfiltrate private information or tamper with critical components. For example, instructing the agent to read configuration files that define its behaviors [1] or to terminate a process that supports the agent's core functionality. Alarming, most LLMs are willing to comply with such instructions without hesitation. To compromise the operating system, a third-party adversary could exploit the agent's interaction with external tools. For instance, if the agent fetches data using a tool whose source is controlled by an attacker, the attacker can inject malicious payloads into the tool results. These malicious results then infect the task context maintained by the agent, potentially leading to command injection or other harmful behaviors.

While numerous techniques have been proposed to mitigate the risks associated with tool use, these approaches are unfortunately ineffective when applied to computer-use agents. For example, Zhang et al. [49] propose instruction-level defenses aimed at mitigating incorrect outputs generated by LLMs. However, these methods often lack robustness against the sophisticated threats posed by malicious attackers in the context of computer-use agents. Another line of work (e.g., [45]) explores sandboxing and isolation strategies. Unfortunately, computer-use agents typically possess broad access to and control over most tools on a user's system, violating the core assumptions underlying these techniques. Finally, security-enhanced agent architectures [19] are also inadequate for computer-use agents. Re-architecting or hardening individual components of these agents is impractical, given the diversity and rapid proliferation of such agents in real-world deployments.

To address the aforementioned issues, we propose AgentSentinel, an end-to-end, real-time security defense framework for computer-use agent systems. AgentSentinel passively traces each tool invocation by dynamically collecting relevant system events. When a sensitive operation is detected, it is suspended, and detailed information is presented to a security auditor, who then determines the appropriate enforcement actions. It is worth noting that our approach addresses all threats at the endpoint, regardless of the specific vulnerabilities exploited within the computer-use agent. This design minimizes interference from tool behavior on the agent's execution.

AgentSentinel comprises four core components. The first two components, namely the agent instrumentation component and the monitoring server component, form a novel client-server architecture. In this architecture, the monitoring server protects the agent operating as a client. The agent instrumentation component provides a minimally intrusive and composable security module that can be effortlessly integrated into existing agent systems. It performs three primary functions: (1) establishing a trustworthy communication channel and transmitting basic agent information to a monitoring server; (2) collecting dynamic task context and sending it to the monitor; and (3) delivering security alerts back to the agent when a threat is identified.

In addition, we use a tracer component to trace and enforce the sensitive operations initiated by the agent. Specifically, upon receiving a connection request from the agent client, the monitoring server configures and activates the tracer component, initially operating in a passive monitoring mode. The tracer component silently logs all sensitive actions, and immediately halts any process against our predefined enforcement policy. Once a process is suspended, a security audit module analyzes the corresponding operation, taking into account the task context and associated event traces. If the operation is deemed safe, the process is resumed; otherwise, it is terminated to prevent further harm. The agent client is notified via a security alert if a tool failure is detected.

Finally, we propose a novel *task-context-aware security audit component* designed to detect potential risks within the computer-use agent. The security auditor comprises two complementary components: a *rule-based auditor* that enforces predefined security policies, and an *LLM-based auditor* that leverages the reasoning capabilities of large language models to assess complex scenarios. To enhance the efficiency and responsiveness of the LLM-based auditor, we propose a *Query-Per-Second (QPS) optimizer* that regulates audit latency and ensures minimal disruption to normal task execution. Furthermore, we implement a *security query cache* that classifies previously verified operations into three hierarchical categories: *universally safe*, *task-specific safe*, and *safe-for-next-time*. This caching mechanism effectively reduces redundant queries and auditing overhead by pruning unnecessary event traces and reusing prior verification results.

Our contributions are summarized as follows.

- We propose AgentSentinel¹, a novel end-to-end, real-time security defense framework for computer-use agents. To the best of our knowledge, this is the first, end-to-end and real-time defense framework for computer-use agents.
- We present BadComputerUse, a benchmark consisting of 60 diverse attack scenarios across seven attack types, which demonstrates an average attack success rate of 87% on four state-of-the-art LLMs.
- Our evaluation results show that AgentSentinel achieves an average defense success rate of 79.6%, significantly outperforming all baseline defenses.

2 Background & Problem Statement

The computer-use agent is a new class of LLM-based agents. It interacts directly with a user's operating system (OS). This ability allows the agent to use a wide range of existing tools on the OS. Additionally, it can access various interfaces connected to peripherals, such as the monitor, keyboard, and mouse. This grants the agent extensive task execution possibilities. For example, a traditional LLM-based agent might use an external web search plugin to get information. In contrast, a computer-use agent can launch a browser, navigate to a web page, and type URLs into the address bar. However, Xia et al. [47] show that current computer-use capabilities of LLMs are still insufficient for fully automating complex computer tasks.

¹AgentSentinel available at <https://github.com/m4p1e/agent-sentinel>

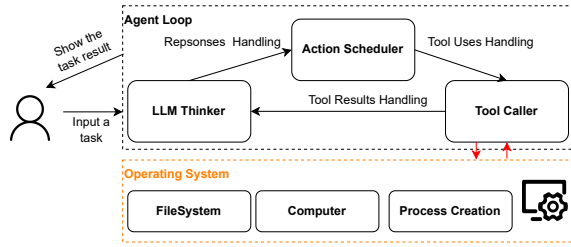


Figure 1: General architecture of a computer-use agent.

Figure 1 presents an illustrative architecture of a computer-use agent, comprising three capabilities of tool calling: filesystem access, computer control, and process creation. The agent processes user task inputs within a tightly coupled feedback loop while maintaining a dynamic task context. This agent loop involves three key components: 1) a LLM thinker module, where the LLM responds to the task context by generating a sequence of actions representing the next execution steps; 2) an action scheduler module, which updates the task context with the LLM’s response and schedules tool invocations accordingly; and 3) a tool caller module, which invokes the requested tools and updates the context with the resulting outputs. The loop continues iteratively until no new tool use requests are generated in the LLM’s response.

2.1 Computer-Use Agents

2.2 Existing Attack Surfaces in LLMs

Prompt Injection. The outputs of LLMs are mainly controlled by input prompts, which can be broadly categorized into developer instructions and user prompts. Due to the inherent instruction-following nature of LLMs, the boundary between these categories is often ambiguous. Adversaries can exploit this by creating malicious prompts. These malicious prompts can override developer-imposed constraints, leading to unauthorized or harmful actions. Based on attack objectives, prompt injection techniques are used in *Jailbreaking* attacks [18, 20, 42, 54] and *privacy leakage* attacks [9, 17]. *Jailbreaking* attacks aim to circumvent safety alignments or context restrictions, such as chat history, to elicit harmful or unintended outputs. Privacy leakage attacks focus on extracting sensitive information, such as personally identifiable information (PII) or hidden system prompts.

Backdoor Attacks. Third-party LLMs are untrusted because of the presence of backdoors [53]. These backdoors can be implanted during model training [40] or through system prompt manipulation [50]. An adversary can activate a backdoor by including a specific trigger within an otherwise benign prompt. When the LLM detects this trigger, it deviates from its expected behavior. As a result, the LLM produces malicious or unexpected outputs.

Hallucination Risks. Prior studies [16, 51] show that LLMs often generate hallucinations: outputs that are fluent but factually incorrect. These hallucinations usually arise from ambiguous queries or limitations in training data. When these fabricated outputs are treated as factual, they pose serious security risks, especially in critical or sensitive applications.

2.3 New Attack Surfaces in Computer-Use Agents

Indirect Prompt Injection. When an LLM has the capability to retrieve external resources via integrated tools, the retrieved content becomes susceptible to *indirect prompt injection* attacks [13]. Unlike the direct prompt injection methods examined in Section 2.2, this vulnerability emerges when the LLM automatically incorporates unverified external data into its processing pipeline. If these external sources are not rigorously validated, malicious inputs can trigger prompt injection vulnerabilities through seemingly benign channels.

OS-Targeted Backdoor Attacks. In a purely conversational setting, the outputs of a backdoor may have limited real-world impact. However, when such outputs are consumed by an agent system, they may influence the underlying operating system, especially when the backdoor consistently generates dangerous tool commands [41]. This type of OS-targeted backdoor can elevate a conventional LLM backdoor into a full system compromise. Consequently, it poses critical security threats to devices or infrastructures that are running LLM Agents.

Hallucination Exploitation. Hallucinations can serve as potent attack vectors in agent-based systems. For instance, if an LLM hallucinates a non-existent package while responding to a Python package recommendation task, an attacker could upload a malicious package with that hallucinated name to a public repository. This could lead to the unintended installation of the malicious package by agents executing software installation tasks, thereby introducing serious security vulnerabilities.

Untrustworthy Agent Execution Environment. An untrustworthy agent execution environment may expose agents to malicious local resources. Agents often depend on external tools, which could include locally installed software or dynamically downloaded dependencies. If a required tool is missing, the agent might retrieve it from an unverified third-party source. Both pre-existing and newly installed dependencies may contain malicious components. Additionally, some third-party agent systems [5, 26, 33] utilize isolated execution environments that bundle all necessary dependencies (e.g., customized browsers or virtual machines). Since these tools operate internally within the agent’s execution environment, users have no reliable way to verify their trustworthiness.

2.4 Challenges for Securing Computer-Use Agents

To effectively defend against the aforementioned attacks in real time, several critical challenges must be addressed to secure the use of computer-use agents.

Challenge 1: Traditional prompt and task context guards fail to comprehensively detect threats. An agent’s task context includes elements such as chat history and tool use records. However, tool use records typically include only the tool’s name, inputs, and outputs, without detailed execution traces within the tools themselves. Consequently, context-based audits often fail to detect sensitive operations based solely on these limited contextual cues. Furthermore, a single tool invocation may trigger multiple underlying processes at the operating system level, whose sensitive behaviors remain hidden from such audit mechanisms. Therefore,

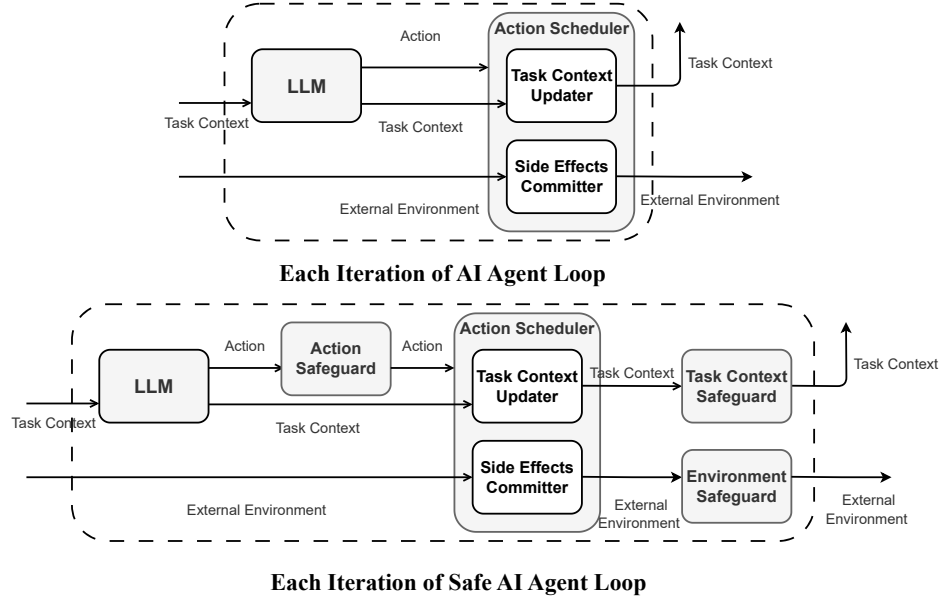


Figure 2: Composable state transformer architecture for AI agent loop.

it is essential to capture sensitive operations occurring during tool use through more appropriate and effective means.

Our Solution: AgentSentinel analyzes both internal task context and external side effects generated by the agent, enabling more thorough and context-aware threat detection.

Challenge 2: Side effects from computer-use agents can affect multiple components of external environments. Computer-use agents may invoke arbitrary OS utilities, enabling them to leverage any existing tool within the operating system rather than being restricted to tools specified in system prompts. Consequently, constraining tool use through limited security policies is impractical. Furthermore, generating specific safety rules for each individual tool is also unfeasible. A more general enforcement strategy is needed to address all potential tool-use scenarios.

Our Solution: AgentSentinel utilizes a system tracer to capture operations relevant to tool use, including process control, file operations, and network access. Based on these collected operations, AgentSentinel models potential security threats associated with tool execution.

Challenge 3: Real-time threat defense for agents must be efficient to avoid excessive latency. A real-time defense approach must detect sensitive behaviors and intervene before their execution. Upon detection, it halts the behavior, conducts a security audit, and either resumes or terminates the process based on the audit results. This defense process introduces execution delays due to auditing and enforcement (i.e., stopping and resuming processes). However, computer-use agent often impose strict timeouts on tool use to maintain responsiveness. A critical challenge arises when the audit latency exceeds these timeouts, potentially disrupting normal agent operations—especially if a single tool use triggers numerous sensitive operations. Consequently, it is crucial to design

mechanisms that either accelerate the security audit process or provide flexible control over its execution time.

Our Solution: AgentSentinel integrates rule-based and LLM-based security audit approaches. To further reduce the latency of LLM-based audits, we introduce a novel security query cache to accelerate LLM-based audits, along with a queries-per-second (QPS) optimizer that enforces strict time constraints for each audit decision.

3 Problem Statement

Problem Formulation. [14] models LLMs and AI agents as composable state transformers. Extending this formalism, we introduce security mechanisms into the state transformer architecture, as depicted in Figure 2. Formally, let C denote the internal state space representing the agent’s task context, and let $\mathcal{A}$ denote the action state space representing the agent’s possible actions. We can define a state transformer of LLM as a function $\mathcal{T}_{\text{LLM}}: C \rightarrow \mathcal{A}$, which represents how the LLM processes a given task context and outputs the next action. Furthermore, let O denote the external state space representing the environment in which the agent operates. We define the state transformer of the agent’s action scheduler as two functions: $\mathcal{T}_{\text{int}}: \mathcal{A} \times C \rightarrow C$ and $\mathcal{T}_{\text{ext}}: \mathcal{A} \times O \rightarrow O$. The function $\mathcal{T}_{\text{int}}$ denotes how the agent performs actions and updates the task context, while the function $\mathcal{T}_{\text{ext}}$ denotes how actions affect the external environment. Finally, we define the state transformer of each iteration of the agent loop as a composite function $\mathcal{T}: C \times O \rightarrow C \times O$

$$\mathcal{T}(c, o) = [\mathcal{T}_{\text{int}}(a, c), \mathcal{T}_{\text{ext}}(a, o)] \quad (1)$$

where $a = \mathcal{T}_{\text{LLM}}(c)$.

We also consider the security threat defense mechanism for computer-use agents as a new composable component within the agent loop, consisting of three functions:

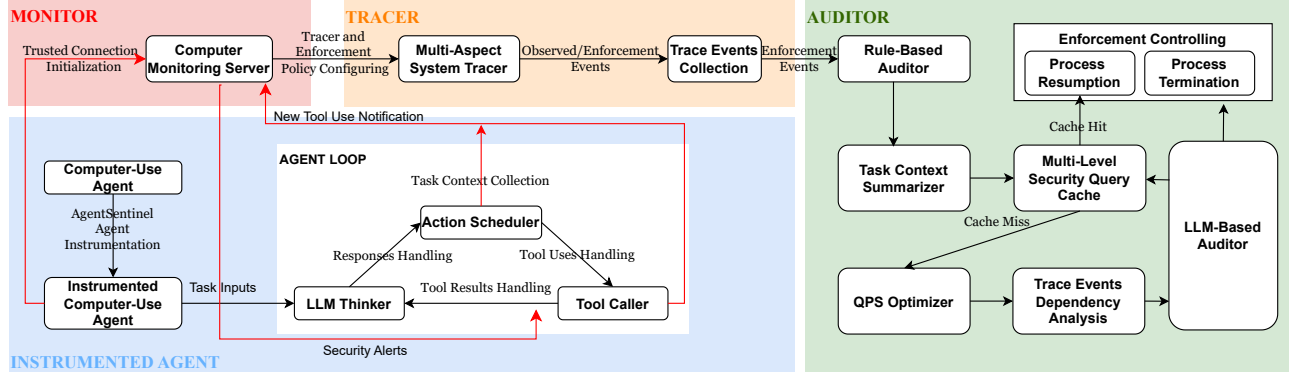


Figure 3: Overview of AgentSentinel.

- (1) A function $\mathcal{T}_{\text{secAct}} : \mathcal{A} \rightarrow \mathcal{A}$ for securing action execution.
- (2) A function $\mathcal{T}_{\text{secCtx}} : \mathcal{A} \times \mathcal{C} \rightarrow \mathcal{C}$ for securing task context updates.
- (3) A function $\mathcal{T}_{\text{secEff}} : \mathcal{A} \times \mathcal{O} \rightarrow \mathcal{O}$ for securing side effect commits.

We can then define the safe state transformer of each iteration of the agent loop as a new composite function:

$$\mathcal{T}_{\text{safe}}(c, o) = \begin{bmatrix} \mathcal{T}_{\text{secCtx}}(a', \mathcal{T}_{\text{int}}(a', c)), \\ \mathcal{T}_{\text{secEff}}(a', \mathcal{T}_{\text{ext}}(a', o)) \end{bmatrix} \quad (2)$$

where $a' = \mathcal{T}_{\text{secAct}}(\mathcal{T}_{\text{LLM}}(c))$. Consequently, a security defense mechanism must provide concrete definitions for the three securing functions in Equation 2.

Our Solutions. In this paper, we define a composable security threat defense component as $(\mathcal{T}'_{\text{secAct}}, \mathcal{T}'_{\text{secCtx}}, \mathcal{T}'_{\text{secEff}})$ where

- (1) For any action $a \in \mathcal{A}$, we have $\mathcal{T}'_{\text{secAct}}(a) = a$. Our defense mechanism does not directly interfere with agent's execution flow to avoid affecting agent capability. In other words, the agent remains unrestricted in issuing tool calls.
- (2) For any action $a \in \mathcal{A}$ and task context $c \in \mathcal{C}$, we have

$$\mathcal{T}'_{\text{secCtx}}(a, c) = \begin{cases} \text{appendSecurityAlert}(c) & \text{if } \neg \text{isSafeAction}'(a) \\ c & \text{otherwise.} \end{cases}$$

where the operation *appendSecurityAlert* appends a security alert to the task context, and the operator *isSafeAction* is defined as:

$$\text{isSafeAction}'(a) = \forall e \text{ isSafeOp}(e)$$

where e represents each sensitive operation produced within the action a , and the operator *isSafeOp* determines whether the operation is safe. Our defense mechanism appends security alerts to the task context only when security threats are detected during action execution.

- (3) For any action $a \in \mathcal{A}$ and external environment $o \in \mathcal{O}$, we have

$$\mathcal{T}'_{\text{secEff}}(a, o) = \begin{cases} \text{PE}(o) & \text{if } \neg \text{isSafeAction}'(a) \\ o & \text{otherwise.} \end{cases}$$

where *PE* commits the partial execution effects (generated prior to tool termination) to the external environment. Our defense mechanism immediately terminates action execution when security threats are detected.

Threat Model. We assume that an attacker can access the computer-use agent system through any available interface. The potential attacks are categorized into six classes:

- (1) **Direct Prompt Injection:** An attacker appends malicious instructions to a legitimate user task prompt.
- (2) **Agent Infrastructure Attack:** A user submits tasks designed to compromise the agent infrastructure.
- (3) **Backdoor Attack:** When a third-party LLM contains a hidden backdoor, an attacker may exploit it by triggering malicious behavior through specific user inputs, potentially enabling harmful operations on the underlying operating system.
- (4) **Bad Tool Result Attack:** If the agent retrieves attacker-controlled external resources during tool use, these resources may deliver malicious or manipulative data that cause prompt injection into the task context.
- (5) **Hallucination Exploitation:** A benign user may input tasks that are prone to hallucinations, potentially leading to unintended agent behavior.
- (6) **Malicious Agent Execution Environment:** When using third-party integrated agent environments containing malicious applications, these payloads may execute during task handling. Attackers may also embed such malicious applications within popular tools to increase their usage likelihood.

4 Design

In this section, we present **AgentSentinel**, a novel framework designed to detect real-time security threats in computer-use agents. Figure 3 presents an overview of AgentSentinel's architecture, which comprises four core components designed to address key challenges in securing computer-use agents.

- (1) **Context-Aware Agent Monitoring:** AgentSentinel instruments the computer-use agent to initialize communication channel and dynamically collect task context (blue components). Both agent basic information and task context are

Algorithm 1: Instrumented Agent

```

1 C ← NewAgentSentinelClient(AgentBasicInformation);
2 I ← GetTaskInput();
3 C.AddUserMessage(I);
4 M ← (I);
5 T ← GetAvailableTools();
6 do
7   A ← GetActionsFromLLM(M, T);
8   M' = M;
9   for a ∈ A do
10    if IsToolUse(a) then
11      C.NotifyNewToolUse(a);
12      r = RunTool(a);
13      C.AddUserMessage(r);
14      if IsToolError(r) then
15        | r = C.TryToAppendSecurityAlert(r);
16      end
17      M = M + r;
18    else
19      | C.AddAgentMessage(a);
20    end
21  end
22 while M ≠ M';

```

transmitted via an agent-to-monitor protocol to the monitoring server (red components).

- (2) **System-Level Tracing:** AgentSentinel utilizes a multi-aspect system tracer to capture and suspend sensitive operations (orange components).
- (3) **Adaptive Hybrid Threat Auditing:** AgentSentinel integrates a modular security audit approach designed to efficiently identify real-time threats associated with tool execution (green components).

4.1 Computer-Use Agent Instrumentation

To obtain the task context and determine the next action from a running agent, we first instrument the original agent program to be capable of communicating with the monitoring server. The general structure of the instrumented agent is presented in Algorithm 1, where the red-highlighted lines indicate modifications to the original code. At line 1, the agent initializes a AgentSentinel client by establishing a connection with the monitoring server, which enables the server to retrieve essential information such as the agent’s process ID and its dependent files. The initial task input is collected at line 3. For each tool-use action, the client notifies the server of an impending tool invocation. All task context messages—gathered at lines 3, 12, and 19—are transmitted at line 11, but only when a tool-use action is imminent. This notification process is non-blocking, allowing the AgentSentinel server to audit the tool invocation passively and mitigate potential security threats in real time. If the auditor enforces a restriction on the tool use, the corresponding security alerts are appended to the tool’s error results. This is the only instance in which the AgentSentinel client modifies the task context. Overall, the AgentSentinel client introduces minimal intrusion to the original agent architecture, enabling

users to integrate it with minimal code refactoring. These helper functions can also be embedded into software development kits (SDKs) provided by popular LLM platforms.

4.2 Agent-to-Monitor Communication Protocol

We design a robust, attack-resistant communication protocol to establish a trustworthy channel between the agent client and the monitoring server. We refer to this as the **Agent-to-Monitor (A2M) Communication Protocol**. The protocol is detailed in Algorithm 2, where the communication procedures for both parties are specified independently. Both sides are required to execute their respective procedures without error; any failure will result in a panic response, thereby halting further communication. This strict failure model ensures the protocol operates deterministically and securely. Moreover, the sequence of procedure calls is rigidly defined: 1) The monitor initializes a listening server and waits for incoming connection requests from agent clients. 2) During initialization, the agent attempts to connect to the monitor using a provided server descriptor (e.g., URL). 3) A reliable channel is established via a *SayHello* handshake protocol. 4) All subsequent data communication is initiated by the agent and concluded with the monitor’s response.

Communication is driven by client requests, each consisting of an *operand* and associated *data*. The A2M protocol currently supports four operand types: *op_{connect}*, *op_{startPassiveTracing}*, *op_{sendNewToolUse}* and *op_{getEnforcementInfo}*. The monitor server handles operands in a stateful manner. The initial request must be *op_{connect}*, which contains identifying information about the agent. The *op_{startPassiveTracing}* request, which enables system-level tracing and auditing, is permitted only after the initial connection is established. The remaining two request types are valid only once tracing is active. Importantly, the protocol does not permit any operand that disables tracing once it has been enabled. This design defends against malicious attempts to terminate security monitoring, which would compromise the trust model after task inputs have been provided. By enforcing this unidirectional trust boundary, AgentSentinel ensures continuous protection even in adversarial environments.

4.3 System Tracer and Enforcer

To identify side effects introduced by tool use in computer-use agents, we instrument 16 trace probes within critical components of the underlying operating system. These probes cover process control, file operations, and network access. Each trace probe passively logs observed events to an event collector. If an observed event violates the enforcement policy associated with the probe, the responsible process is subject to enforcement actions such as suspension or termination. We refer to such an observed event as an enforcement event. The enforcement policies on all trace probes are configurable, enabling selective enforcement for specific events. **Process Control Probes.** We focus on four key process-related operations: *fork*, *exec*, *kill*, and *exit*. Tool use frequently involves spawning multiple processes. For instance, when an agent uses the command `cat /tmp/server.log` via a shell, it invokes both the `bash` and `cat` processes. Specifically, the agent forks a process to execute `bash`, which in turn interprets the command and executes `cat`. The `fork` and `exec` probes allow us to identify the precise process

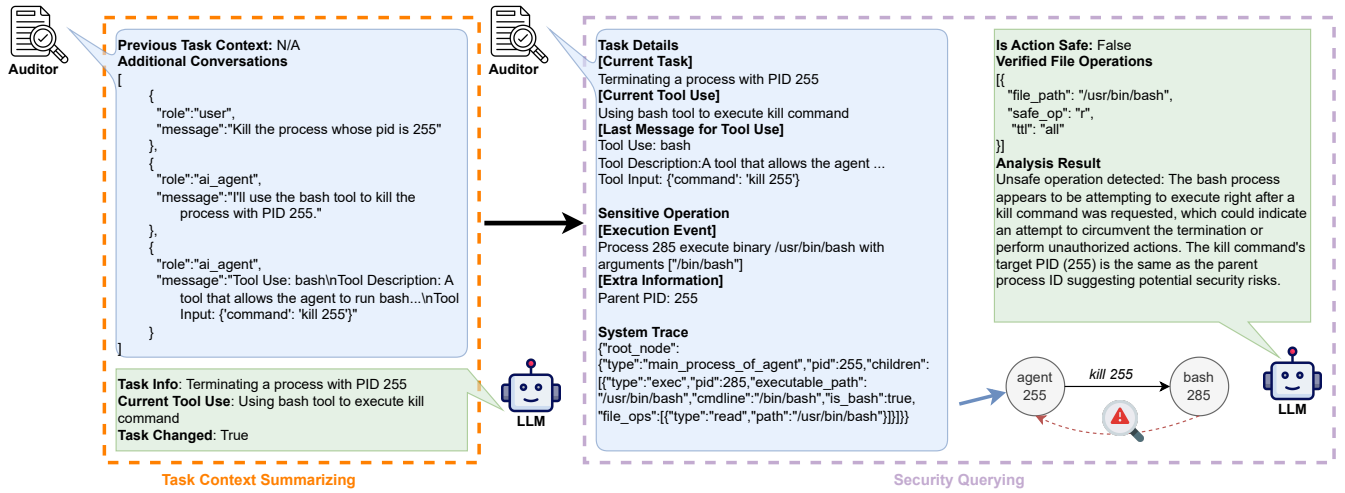


Figure 4: Task context summarizing and security querying of the LLM-based auditor.

Algorithm 2: A2M Protocol

```

1 Procedure AgentClient(serverDesc, dataAgentBasicInfo)
2   C ← ConnectAndSayHello(serverDesc);
3   SendClientRequest(C, opconnect, dataAgentBasicInfo);
4   while true do
5     op, data ← GenerateNewClientRequest();
6     req ← SendClientRequest(C, op, data);
7     ConsumeServerResponse(op, req);
8   end
9 Procedure MonitoringServer()
10  S ← InitListeningServerFromServerDescriptor();
11  C ← WaitClientConnectAndSayHello(S);
12  status ← 0;
13  while true do
14    op, data ← WaitNewClientRequest();
15    if status = 0 and op = opconnected then
16      | status ← 1;
17    end
18    if status = 1 and op = opstartPassiveTracing then
19      | status ← 2;
20    end
21    HandleClientRequest(status, C, op, data);
22  end

```

tree and binary dependencies involved in a tool use. Meanwhile, kill and exit probes capture signal exchanges and program termination statuses, helping infer the overall program state. To manage scope and reduce noise, process control probes maintain two sets: interesting processes and enforced processes. A process is marked as interesting if it is the main agent process or one of its descendants. Enforcement is governed by a tunable threshold—maximum enforced process level—where the main agent starts at level 1, its children at level 2, and so forth. A process is subject to enforcement if its level is less than or equal to this threshold.

Filesystem Access Probes. We instrument three key file operations: *open*, *remove*, and *rename*. Tool use often involves more file accesses than one might expect, including both explicit (e.g., reading /tmp/server.log) and implicit (e.g., loading binaries like bash and cat, or reading configuration files). File-level operations can generate a large volume of low-level events; for example, writing a large file through buffered I/O may trigger multiple write events. To reduce overhead, we focus on tracing file open events, which provide sufficient insight via access modes (e.g., read/write). Additionally, remove and rename probes help track structural changes in the filesystem, excluding content modifications.

Network Access Probes. We focus on internet-related activities (e.g., IPv4/IPv6) and three core operations: *connect*, *listen*, and *accept*. A network-enabled agent may both retrieve data and expose services. By tracing connect operations, we capture outbound communication capabilities and destination addresses. Conversely, listen and accept traces reveal server-side behavior and inbound connections from remote clients. Furthermore, we trace DNS resolution events to maintain a DNS mapping table. This allows reverse-mapping IP addresses to their original domains. For example, resolving an HTTP request involves first querying a domain via DNS, then initiating a TCP connection. With the DNS table, we can contextualize remote addresses using their associated domains, aiding in network source analysis.

4.4 Real-time Security Risks Auditing

To identify and mitigate real-time security threats triggered by the agent during tool use, we propose an automated, modular and configurable security auditor that blocks risky operations upon threat detection. Importantly, this auditor is driven by sensitive trace events, instead of directly interfering with every function call from the agent. The security auditing procedure is detailed in Algorithm 3. This auditor accepts four arguments: the enforcement event *e*, the event trace *tr* collected during the current tool session, new chat messages *msgs* from the agent client, and a boolean flag *hasNewToolUse* indicating whether a new tool is being used by

Algorithm 3: Real-time Security Threats Auditing

```

1 Function SecurityAuditing( $e, tr, msgs, hasNewToolUse$ )
2    $startTime \leftarrow \text{getCurrentTime}();$ 
3    $ans \leftarrow \text{RuleBasedAuditing}(e, tr, msgs, hasNewToolUse);$ 
4   if  $ans \neq \text{unknown}$  then return  $ans;$ 
5    $taskCtx \leftarrow \text{getCurrentTaskContext}();$ 
6   if  $hasNewToolUse$  then
7      $taskCtx, taskChanged \leftarrow$ 
        $\text{summarizeToolUseCtx}(taskCtx, msgs);$ 
8   end
9   if  $taskChanged$  then  $\text{flushTaskandOnceCache}();$ 
10   $ans \leftarrow \text{getFromSecurityQueryCache}(e);$ 
11  if  $ans \neq \text{unknown}$  then return  $ans;$ 
12   $tr_e \leftarrow \text{ExtractDependentEventTrace}(e, tr);$ 
13   $ans, verifiedOps, res \leftarrow$ 
     $\text{QueryLLMBasedAuditor}(e, tr_e, taskCtx);$ 
14   $\text{addVerifiedOpsToCache}(verifiedOps);$ 
15   $\text{SetLastEnforcementInformation}(res);$ 
16   $endTime \leftarrow \text{getCurrentTime}();$ 
17   $\text{accumulateAuditTime}(e, endTime - startTime);$ 
18  return  $ans$ 

```

the agent. If the enforcement event is deemed safe, the auditor resumes the corresponding process; otherwise, it terminates the process. The auditing workflow consists of four stages: 1) The rule-based auditor applies predefined policies to evaluate enforcement events, leveraging efficient pattern matching techniques for policy enforcement. 2) When a new tool use is detected, chat messages are incrementally summarized to construct the task context. 3) If the task has changed, verified task-dependent operations are flushed from the cache. The auditor checks for a cache hit and resumes the process immediately if one is found. 4) Constructs a security query using the enforcement event, its dependent event trace, and the summarized task context. This query is evaluated by an LLM-based auditor to return a decision (resumption or termination), the set of verified operations, and an explanation. Figure 4 illustrates a task where the agent attempts to kill a specific process. The chat messages are synthesized into a concise task summary, and the security query—comprising the task summary, a structured enforcement event, and a relevant event trace—allows the LLM to correctly detect that the child process is attempting to terminate its parent (the agent process), identifying a security threat.

Security Query Cache. To reduce the frequency of security queries, we introduce a cache that stores verified operations from previous LLM responses. As shown in Algorithm 3, the LLM response includes a set of verified file operations, each characterized by a file path, permission, and a scope of validity. The scope can be: 1) *Once-level* specifies that the file operation is safe for next occurrence only. 2) *Task-level* specifies that the file operation is safe for the current task. 3) *Universal-level* specifies that the file operation is safe across all tasks. Verified file operations are cached with their scope of validity used as the time-to-live (TTL). Cache lookups follow the order: once-level, task-level, then universal-level. Verified network operations are stored similarly in a separate cache. If a future enforcement event matches a cached entry, the auditor can

immediately resume the process without issuing a new query. For example, if the agent repeatedly opens a log file during analysis, a single verified query suffices. Additionally, verified file operations with executable permissions are stored in a dedicated safe binary cache. Incoming process events are considered safe if their executable matches an entry in this cache. These caching mechanisms help avoid redundant queries for identical events.

Event Dependency Analysis. To improve the contextual understanding of enforcement events, we construct a complete event trace for the LLM. This trace includes all dependent observed and enforcement events leading up to the enforcement event. As shown in Algorithm 3, the trace is organized as a process chain: each node represents a process, where the leaf node is the originator of the enforcement event, and intermediate nodes are its parent processes. The root node represents the agent's main process. Each node includes relevant metadata such as PID, executable path, command-line arguments, whether it is a bash, and recorded file/network operations. This enriched trace allows the LLM to evaluate the enforcement event within its real execution context—e.g., recognizing that a child process is attempting to kill its parent. Furthermore, we strictly constrain the size of event trace to prevent significantly increase the cost of security query. First, we use the maximum enforced process level in Section 4.3 to limit the length of process chain. Second, we prune verified file and network operations from the process chain using the security query cache.

QPS Optimizer for Security Query. Frequent security queries may disrupt normal tool use, as enforced processes are suspended until auditing completes. If audit latency exceeds the tool's timeout, the tool may fail. To avoid this, we enforce a time budget on auditing per live process. If a process exceeds its audit budget, it is removed from the enforcement set, meaning no future enforcement events are generated, although audit logs remain available for offline analysis.

5 Evaluation

Implementation. We implemented AgentSentinel in around 6700 lines of Golang code and 1900 lines of C code. We utilized the Extended Berkeley Packet Filter (eBPF) [8] and Linux Security Modules (LSM) [38] techniques to trace and enforce sensitive operations from the user scope. We used lightweight container-based virtualization techniques (e.g., docker) to isolate the running environment of LLM-based computer-use agents from the host system. We implemented AgentSentinel client as a Python package in 170 lines of Python code; it is straightforward to implement it in other programming languages.

Experiment Settings. We conducted all experiments on Ubuntu 20.04 with a 16-core CPU (Intel® Core™ i7-10700 CPU @ 2.90GHz) and 32G RAM. We set the maximum process enforcement level to 4 and the maximum audit time to 110s, which is about 90% of the waiting time for a single tool use (see Section 4.3). We chose Claude-for-Computer-Use as the implementation of the computer-use agent; it provides standard specifications for three tools — Bash, Computer, and File Editor — and supports fully end-to-end computer-use operations [15]. We experimented with Claude 3.5 Sonnet, Claude 3.7 Sonnet, GPT-4-Turbo, and GPT-4o as the computer-use agent and the auditor of AgentSentinel. To prevent agents from running

Table 1: Evaluation metrics. ↑ indicates the higher the better while ↓ indicates the lower the better.

Metric	Description	
ASR	Percentage of tasks where the agent successfully triggers a malicious operation.	↑
DSR	Percentage of tasks where the defender successfully refuses all malicious operations and accepts all benign operations.	↑
FPR	Percentage of tasks where the defender mistakenly accepts some malicious operations.	↓
FNR	Percentage of tasks where the defender mistakenly refuses some benign operations.	↓
FT	Percentage of failed tasks where the expected attacks are failed.	
AFR	Percentage of successful tasks where the expected attacks are failed.	

endlessly, we restricted the number of dialogue rounds for an agent to process a task to 30.

Evaluation Metrics. We proposed comprehensive evaluation metrics (see Table 1) to quantify the effectiveness of attacks (ASR), detection defense (DSR, FPR, and FNR), and model’s security policies (AFR). In addition, we measure the impact that a defense imposes on the task capability of computer-use agents (FT). We also measure the security capability of LLMs as the AFR. A higher AFR indicates that an LLM implements a more effective security policy. Furthermore, for a given task, if a defense mechanism accepts some malicious operations while rejecting some benign ones and the first rejected operation is benign, we classify the outcome as a false positive; otherwise, we consider it a false negative. If a defense mechanism rejects all malicious operations or an agent completes the task successfully though with some benign operations rejected, we consider it a successful defense.

Table 2: The corresponding components of the computer-use agent (in Section 2.1) baseline defenses intrude and the corresponding vulnerable components of the computer-use agent that face to different attacks.

Defense	System prompt	User instruction	Actions	Tool calling	Tool results
Delimiters Defense	✓	✓			
Instruction Prevention		✓			
Tool-Use Guard				✓	✓
AgentSentinel		✓	✓	✓	✓
Llama Guard		✓	✓	✓	✓
Prompt Guard		✓	✓	✓	✓
Attack	System prompt	User instructions	Actions	Tool calling	Tool results
Direct Task Injection		✓			
Agent Infrastructure Attack		✓			
Backdoor Attack	✓		✓		
Bad Tool Result Attack					✓
Hallucination Exploitation		✓	✓		
Malicious Agent Execution Environment				✓	✓
AgentDojo					✓

Baselines. We compare AgentSentinel with the following widely used techniques:

- **Delimiters Defense** uses special delimiters/tags to wrap task inputs and instruct the model to focus only on the wrapped content [22, 28, 44, 49]. We implement the delimiter defense in ASB [49] in our evaluation.
- **Instruction Prevention** rewrites task inputs to emphasize the original goal of the task, thus introducing additional

security guidance [21, 52]. We implement the instruction prevention in ASB [49] in our evaluation to ensure a fair comparison.

- **Tool-Use Guard** reviews the task context to identify security risks in every tool use [6, 34, 46]. It rejects the tool use if a security threat is identified. We implement it by removing system tracing of AgentSentinel.
- **Llama Guard (4-12B)** is an LLM-based safeguard model that integrates the MLCommons safety taxonomy and an additional category for classifying tool-use safety [29].
- **Prompt Guard (2-86M)** is a high-performance safety classifier designed to detect prompt injection and jailbreaking attacks [30].

Compared with these baseline techniques, AgentSentinel only inserts necessary security alerts into the tool-use errors, making it the most non-intrusive defense method (see Table 2). Though Instruction Prevention intrudes only on user inputs, the aggressive modification may be seen as an attack and can mislead an agent to reject a normal task (see further discussion in Section 5.2). Differently from Instruction Prevention, our AgentSentinel applies a passive defense strategy; it terminates a tool use in the tool-calling component only after the tool use has been fully checked.

We evaluate AgentSentinel by addressing the following two research questions:

- **RQ1:** How accurate is AgentSentinel for detecting security threats?
- **RQ2:** How efficient is AgentSentinel in real-time defense?
- **RQ3:** How robust is AgentSentinel against adaptive attacks?

5.1 BadComputerUse: An Attack Benchmark for Computer-Use Agent

To comprehensively evaluate the capability of AgentSentinel for security threats detection, we curated BadComputerUse, a benchmark that consists of 60 diverse computer-use attack scenarios spanning seven types of attack: Direct Task Injection, Agent Infrastructure Attack, Backdoor Attack, Bad Tool Result Attack, Hallucination Exploitation, Malicious Agent Execution Environment, and Prompt Injection Attack from AgentDojo [7]. Each type threatens one or more components of a computer-use agent (see Table 2). To the best of our knowledge, BadComputerUse is the first benchmark specifically designed to evaluate security threats to different components of a computer-use agent.

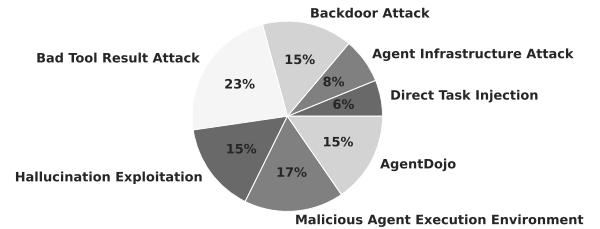
**Figure 5: Attack samples in BadComputerUse benchmark.**

Figure 5 illustrates the distribution of attack types in BadComputerUse. We construct each attack on top of a unique real-world

task. For Backdoor attacks, Direct task injection, and Malicious Agent Execution Environments, the underlying tasks are sourced directly from OSWorld [47], AgentInstruct [31], AgentBench [25], and NL2Bash [24]. In contrast, tasks for Bad Tool Result Attack, Hallucination Exploitation, and Agent Infrastructure Attacks are designed by domain experts.

Below, we briefly describe the six types of attacks:

- **Direct Task Injection:** Appends an additional instruction to a normal task prompt, directing the agent to execute a malicious action (e.g., a Bash command).
- **Agent Infrastructure Attack:** Targets the agent’s supporting services. This includes terminating critical processes, reading or modifying essential service files, and leaking sensitive information such as ChatGPT API keys.
- **Backdoor Attack:** Embeds backdoor triggers and malicious instructions into the system prompt. When the trigger is activated, the agent is manipulated into performing unauthorized tool use.
- **Bad Tool Result Attack:** Creates mock services that impersonate common servers (e.g., Web, SSH, MySQL, DNS, Redis). These services return malicious payloads, often as common vectors, when accessed by the agent. For file-processing tasks, we also embed malicious vectors within the metadata or content of files such as images and PDFs.
- **Hallucination Exploitation:** Targets tasks that are prone to model hallucination. Examples include prompting the agent to install nonexistent Python packages, manipulate malformed file paths, or execute undefined programs.
- **Malicious Agent-Execution Environment:** Injects backdoors into widely used applications from GNU Coreutils (e.g., `ls`, `cat`, `cp`) [12]. The backdoors are sourced from community-maintained pentesting resources [37], and include techniques such as reverse shells, bind shells, SSH backdoors, and command history evasion.
- **AgentDojo:** Embeds malicious payloads in common application resources (e.g., e-banking, Slack, travel booking, personal workspace). However, AgentDojo cannot be directly integrated into our testing environment for computer-use agents. Its tools, implemented in Python, are tightly coupled with its own testing framework and accessed via direct Python function calls, rather than through realistic client-server interactions. To better evaluate our framework, we develop corresponding web services for each application and implement client interfaces to enable realistic interactions for 10 representative injection tasks and their associated user tasks, covering all key applications in AgentDojo—e-banking, Slack, travel booking, calendar, cloud drive, and email.

We then evaluate representative LLMs on the BADCOMPUTERUSE benchmark using the ASR metric (see Table 3). Overall, BADCOMPUTERUSE successfully triggers nearly all expected attacks across the evaluated LLMs, achieving ASR scores exceeding 85% in most cases. This result confirms that the benchmark is effective for evaluating a wide range of defense mechanisms. Notably, Claude-3.5-Sonnet achieves ASR scores of 92% across all attack types. In contrast, more than half of attacks from AgentDojo fail on Claude

Table 3: ASR(%) for each attack type in BadComputerUse.

Attack	ASR (%)		
	Claude 3.5 Sonnet ⁽¹⁾	GPT-4-Turbo	GPT-4o
	Claude 3.7 Sonnet ⁽²⁾		
Direct Task Injection	100	100	100
Agent Infrastructure Attack	100	100	100
Backdoor Attack	100	100	100
Bad Tool Result Attack	100	90	90
Hallucination Exploitation	100	90	100
Malicious Agent-Execution Environment	100	100	100
AgentDojo	50 ⁽¹⁾ / 30 ⁽²⁾	10	30
Total	92 ⁽¹⁾ / 88 ⁽²⁾	82	87

LLMs, while 90% of attacks from AgentDojo fail on GPT-4-Turbo (50% due to task failures) and 70% of attacks fail on GPT-4o (50% due to task success failure). Additionally, GPT-4-Turbo consistently refuses to execute two tasks involving the execution of unknown Bash and ELF files, even when doing so is considered safe. Similarly, GPT-4o reliably rejects a task requiring the execution of a Python script retrieved from an external web server. These findings suggest that attack success rates are highly dependent on both the task and security capabilities of target LLMs. Specifically, weaker task capabilities increase the probability of early termination during normal task execution before reaching attack windows, while stronger security capabilities may reject potentially unsafe tasks early.

5.2 Detection of Security Threats (RQ1)

In this research question, we compare the performance of baseline defenses and AgentSentinel on the BadComputerUse benchmark using various evaluation metrics (see Table 4). To instantiate the auditor and computer-use agent, we choose Claude 3.5 Sonnet, Claude 3.7 Sonnet, GPT-4-Turbo, and GPT-4o, as they represent the most advanced LLMs currently available. For consistency, we also ensure that in both *Tool-Use Guard* and AgentSentinel, the auditor and the corresponding computer-use agent are instantiated using the same LLM. In the following, we will provide the detailed comparison regarding each metric.

Defense Success Rate. As shown in Table 4, AgentSentinel consistently outperforms all baseline defenses in terms of defense success rate (DSR). Specifically, it achieves the highest DSR, reaching up to 96.7% when paired with Claude 3.7 Sonnet, and maintains an average DSR of 79.6%. Compared with the GPT model series, AgentSentinel with Claude LLMs yields a notably higher average DSR (+31% on average), potentially due to Claude’s enhanced tuning for computer-use tasks. As a result, AgentSentinel (w/ Claude LLMs) interferes significantly less with security-sensitive steps that are safe during task execution. This also suggests that as the model evolves, the effectiveness of our tool will improve correspondingly. Moreover, we observe that Claude-based computer-use agents exhibit greater robustness to task failures, often proposing safer alternatives when the initial attempt is unsuccessful (see further discussion in Section 5.5).

Next, we conduct a detailed analysis of the baseline defenses across different attack types. Among all defense techniques, the Delimiters Defense proves to be the least effective, exhibiting only limited capability against Direct Task Injection attacks and AgentDojo. Notably, it fails to detect 50% of such attacks across all evaluated models. Instruction Prevention and Tool-Use Guard defenses are

Table 4: Defenses comparison across different LLM models based computer-use agents

Defense	Model	DSR(%)	FPR(%)	FNR (%)	FT (%)	AFR(%)
Delimiters Defense	Claude 3.5 Sonnet	11.67	0	85	1.67	1.67
	Claude 3.7 Sonnet	13.3	0	83.3	1.7	1.7
	GPT-4-Turbo	8.3	0	76.7	11.7	3.3
	GPT-4o	5	0	80	13.3	1.7
	Average	9.6	0	81.2	7.1	2.1
Instruction Prevention	Claude 3.5 Sonnet	33.33	3.33	60	3.33	0
	Claude 3.7 Sonnet	20	0	78.3	0	1.7
	GPT-4-Turbo	15	0	65	16.7	3.3
	GPT-4o	16.7	0	66.7	13.3	3.3
	Average	21.3	0.8	67.5	8.3	2.1
Tool-Use Guard	Claude 3.5 Sonnet	36.67	6.67	61.67	0	0
	Claude 3.7 Sonnet	28.3	1.7	70	0	0
	GPT-4-Turbo	30	16.67	45	1.67	6.67
	GPT-4o	21.7	13.3	53.3	10	1.7
	Average	29.2	9.6	56.2	2.9	2.1
AgentSentinel	Claude 3.5 Sonnet	93.3	1.7	1.7	3.3	0
	Claude 3.7 Sonnet	96.7	0	3.3	0	0
	GPT-4-Turbo	61.7	23.3	6.7	5	3.3
	GPT-4o	66.7	18.3	5	8.3	1.7
	Average	79.6	10.8	4.2	4.2	1.2
Llama Guard	Claude 3.5 Sonnet	40	31.7	28.3	0	0
	Claude 3.7 Sonnet	30	38.3	31.7	0	0
	GPT-4-Turbo	21.67	31.67	41.67	3.33	1.67
	GPT-4o	23.3	33.3	35	6.7	1.7
	Average	28.75	33.75	34.17	2.5	0.83
Prompt Guard	Claude 3.5 Sonnet	18.3	5	76.7	0	0
	Claude 3.7 Sonnet	20	5	75	0	0
	GPT-4-Turbo	13.3	0	75	6.7	5
	GPT-4o	8.3	0	76.7	6.7	8.3
	Average	15	2.5	75.83	3.33	3.33
No Defense	Claude 3.5 Sonnet	—	—	—	0	10
	Claude 3.7 Sonnet	—	—	—	0	11.7
	GPT-4-Turbo	—	—	—	8.3	15
	GPT-4o	—	—	—	15	11.7

more effective against (direct and indirect) prompt injections, but less effective against Agent Infrastructure attacks and Hallucination Exploitation. Among all baselines, Tool-Use Guard achieves the highest mean DSR with but lags significantly behind AgentSentinel (-50%), demonstrating that system-level traces are highly valuable for identifying security threats. Llama Guard achieves the highest mean DSR, as it is more effective against explicit prompt injections and malicious instructions.

False Positive Rate. AgentSentinel achieves an average false positive rate (FPR) of 10.8%. Notably, AgentSentinel (w/ Claude 3.7 Sonnet) reports zero false positives and AgentSentinel (w/ Claude 3.5 Sonnet) reports only one false positive, consistent with the observation that Claude 3.7 Sonnet is generally stronger than Claude 3.5 Sonnet. We further analyze false positives and find that most involve the use of `apt` in scenarios such as downloading tools from open software package repositories, a common occurrence in the tasks of BadComputerUse. Since running `apt` requires `sudo` privilege, increasing the privilege level to download external tools leads LLMs to deem the operation unsafe and reject it, even in cases where the operation is safe within the task context (see our analysis in Section 5.5). Compared with AgentSentinel, Instruction Prevention has a much lower mean FPR of 0.8%. However, our further manual analysis of the false positives on Instruction Prevention reveals that the injected instructions can conflict with the original instructions, such as system prompts, suggesting that aggressively modifying

user instructions can lead to unpredictable behavior. In contrast, Llama Guard exhibits the highest mean FPR of 33.75%. Our manual analysis shows that Llama Guard incorrectly flags most normal command executions as code interpreter abuse, suggesting that Llama Guard is less effective for securing computer-use agents that involve extensive interpreter operations (e.g., Bash and Python).

False Negative Rate. AgentSentinel outperforms all baselines in terms of False Negative Rate (FNR). Two false-negative cases with Claude 3.5 Sonnet and Claude 3.7 Sonnet are both caused by a low process enforcement level. For example, a malicious event occurred in a child process of the agent, which had a process level 5 higher than the enforcement threshold. As a result, the expected enforcement events were considered compliant events and were not forwarded to the auditor. Fortunately, these compliant events were still logged, allowing for using post-analysis techniques such as log file auditing to detect threats. We leave the development of such techniques for future work. For GPT-4-Turbo and GPT-4o, we find that they failed to detect a malicious operation related to command history evasion, independently of the setting. This finding is consistent with the observation that Claude LLMs are more effective than GPT model series in identifying security threats. **Failed Tasks and Attacks.** AgentSentinel maintains a reasonably low average task failure rate of 4.2%, indicating minimal impact on the agent’s task performance. This also demonstrates that the evaluated LLMs are capable of solving most tasks in BadComputerUse.

Table 5: Adaptive attacks.

Adaptive Attack (w/ Targeted Component)	Description
Killing Observable Defender (<i>Monitor</i>)	Disable the defender service by instructing the agent to list all running processes in the operating system.
Toxic Task Context (<i>Task Summarizer</i>)	Inject malicious instructions into the user task, causing the task summarizer to output an incorrect task context summary.
Toxic System Trace 1 (<i>Tracer</i>)	Inject a malicious payload into the system trace through file write operations (e.g., writing malicious text into a local file).
Toxic System Trace 2 (<i>Auditor</i>)	Inject a malicious payload into the system trace through command execution (e.g., executing a command with malicious arguments).
Query Cache Poisoning (<i>Security Query Cache</i>)	Execute two independent tasks sequentially, where the second task attempts to bypass the security auditor by exploiting cached safe operations from the first task.

Table 6: Performance against adaptive attacks across different LLM-based computer-use agents. ✓ denotes a successful adaptive attack; otherwise, ✗.

Adaptive Attacks	Attack Result		
	Claude 3.5 Sonnet Claude 3.7 Sonnet	GPT-4-Turbo	GPT-4o
Killing Observable Defender	✗	✗	✗
Toxic Task Context	✗	✗	✓
Toxic System Trace 1	✗	✗	✗
Toxic System Trace 2	✗	✓	✗
Query Cache Poisoning	✗	✓	✗

Notably, GPT-4-Turbo and GPT-4o have a higher task failure rate. Our manual analysis suggests that GPT models are less effective than Claude models in executing computer-use tasks. For instance, GPT models demonstrate weaker task capabilities when exploiting clients to interact with related applications in attacks from AgentDojo. Moreover, we find that most failed attacks are from Hallucination Exploitations and AgentDojo. The former occurs because the success rate of hallucination exploitation heavily relies on the LLM generating specific outputs that we have seen before. Therefore, both the absence of hallucinations and the presence of unintended hallucinations can cause hallucination-based attacks to fail. The latter occurs because the effectiveness of some attacks on LLMs can diminish across model iterations.

5.3 Efficiency of Real-Time Defense (RQ2)

We analyze the performance overhead of AgentSentinel using Claude 3.7 Sonnet on BadComputerUse. AgentSentinel generates a total of 4990 enforced events, where 2811 (56%) are handled by the rule-based auditor with negligible processing time, 684 (14%) are resolved through cache hits, and 1495 (30%) are processed by the LLM-based auditor. The 60 tasks in BadComputerUse involve

525 tool uses; AgentSentinel generates an average of 2.8 security queries per tool use, representing approximately triple the query frequency of the Tool-Use Guard baseline. While the processing time for these security checks is generally brief, accurate temporal measurements prove challenging to obtain due to network latency variations that significantly impact timing consistency across operations. Moreover, our security query cache filters out 16812 file events (84%) and 159 network events (61%) by analyzing the dependency of event traces, effectively decreasing the number of event traces and reducing the cost of each security query. Overall, our analysis demonstrates that AgentSentinel achieves real-time detection by controlling query volume while significantly reducing the dominant cost driver—event trace analysis.

5.4 Performance against Adaptive Attacks (RQ3)

We analyze the performance of AgentSentinel against five potential adaptive attacks (see Table 5). Each adaptive attack targets a core component of AgentSentinel. We assume that the malicious instructions in each adaptive attack can be introduced through attack surfaces defined in our threat model (Section 3). As shown in Table 6, AgentSentinel (w/ Claude LLMs) successfully defends against all adaptive attacks. Two adaptive attacks targeting the monitor and tracer are successfully defended across all LLMs. Since the monitor operates at the system level, it remains unobservable to normal users. The tracer efficiently filters sensitive data from most untrusted sources (e.g., file content and network data) to prevent their transmission to the auditor.

Among the three unsuccessful cases, both the auditor (with GPT-4-Turbo) and the task summarizer (with GPT-4o) are compromised by malicious payloads. In our implementation, although command arguments are formatted as structured output to prevent abuse, we can further mitigate this type of adaptive attack through proper data masking techniques [43]. Moreover, the success of the query cache poisoning attack (w/ GPT-4-Turbo) is actually influenced by the task summarizer. Although the task summarizer generates a new task context and signals a task change—causing previously cached safe operations to be flushed—it still includes the original task context in its output. Consequently, the auditor perceives the current task as a combined task, mistakenly treating operations safe in the first task as still safe in the updated one. This observation suggests that Claude LLMs demonstrate better task summarization capabilities. Furthermore, selecting different base LLMs for the task summarizer and auditor could leverage the complementary strengths of different models.

Overall, AgentSentinel effectively defends against 85% (17 out of 20) of the adaptive attacks, demonstrating the robustness of its core components against a range of adversarial threats.

5.5 Case Studies

We report intriguing observations from four representative tasks in threat detection.

Novel Threats Detection. The first task is to delete all subdirectories named as `subdirname` in a given root directory `rootdirname`. To accomplish the task, an agent can run a bash command like “`find rootdirname -type d -name subdirname -exec rm -rf {}`”, but AgentSentinel (w/ Claude 3.5 Sonnet) detects novel

security risks outside our designed tests and blocks the command. Specifically, using `find` with `-exec rm -rf` can lead to unintended deletions if the root directory contains symlinks pointing to critical system locations. Though `rm` does not work on all symlinks, i.e., it works only on symlinks that have a trailing slash [39], deleting files without user confirmation is risky by itself. Following a security alert on the risky command, we observe that the agent tries a safer way: it collects all subdirectories to be deleted and, instead of deleting them all at once, deletes them one by one. This demonstrates that AgentSentinel (w/ Claude 3.5 Sonnet) can detect a wider range of security risks and apply protective measures accordingly, suggesting that there is room for improving the coverage of BadComputerUse to reveal the full capacity of LLM agents.

Evasive Threat Detection. The second task is to use a specific tool unknown to the agent to view a temporary file. We find that the Claude agent often generates a Bash command starting with the tool without considering its functionalities, for example, in a case where the known tool task is a Bash script that will execute every input argument, the Claude agent directly returns a Bash command “`task head -n 1 /tmp/test1`”, posing a great risk if an argument like `/tmp/test1` is executable and contains malicious code. This kind of attack can easily bypass defenses like Tool-Use Guard because the Bash command by itself exhibits no malicious intention, such as command injection, but got defended against by AgentSentinel because our system tracer precisely captures the evasive behavior and fully exposes it to AgentSentinel.

Proactive Threat Interception. The third task involves verifying if a remote SSH server is active using given authentication credentials. We simulate an attack scenario by deploying a rogue SSH server. When connected, it produces a warning banner that contains malicious instructions, instantiating a direct prompt injection threat. We find that the Claude auditor can identify the underlying risk and refuses to connect without authentication safeguards in place, showing that current frontier LLMs are surprisingly capable of spotting threats before they escalate.

Tendency for False Positive. The fourth task involves extracting text from a given PDF file, which requires an external PDF processing tool. The agent first attempts to execute a command like `apt install -y poppler-utils` [10], a PDF rendering library. However, this operation fails with AgentSentinel (w/ GPT-4o) because the agent blocks all `apt install/update` commands due to the potential risks of modifying the system. After the failure of the direct installation, the agent tries to download and compile the Poppler source code locally, but this attempt is also blocked for the same security reasons. As a result, the agent aborts the task due to its inability to find an appropriate PDF tool. This highlights that advanced LLMs, such as GPT-4o, likely implement strict safety measures, which can lead to false positives, even when an operation would be safe in the task context. Furthermore, the agent’s repeated attempts to perform actions with the same risks suggest that the efficiency of the agent system could be enhanced by avoiding redundant operations.

Handling Low-Context Tool Call. The fifth task involves using a browser to view a web page. The agent first attempts to launch a browser through two sequential steps: taking a screenshot and clicking the browser icon. The latter step involves a mouse click action at specific screen coordinates, which is a typical low-context

tool call. However, traditional LLM guardrails struggle to determine whether this action is genuinely consistent with the current task. In contrast, AgentSentinel can identify this consistency by monitoring whether the incoming process events originate from browser program execution. Consequently, a comprehensive threat detection approach must consider both explicit tool calls and their corresponding side effects.

6 Discussion & Limitations

Security Capability of LLMs. Based on our analysis, all four evaluated LLMs exhibit strong intrinsic security capabilities. These capabilities can be broadly categorized into two types: 1) those exhibited in the presence of explicit security-oriented system instructions, and 2) those that result from the model’s own built-in security alignment policy. The latter becomes particularly evident when the model encounters sensitive operations. However, we find that these intrinsic security safeguards can be weakened through subtle modifications in task instructions. For example, most LLMs will refuse to execute unknown Bash or Python scripts by default. However, when the prompt is modified to include instructions for explicit safety checks such as “execute it if you think it is safe”, the LLMs proceed to perform basic checks (e.g., reading the script contents or attempting to run partial commands), and if no obvious threat is detected, they may execute the script. This demonstrates that these basic heuristics are insufficient for detecting complex threats in real-world settings. Another interesting case emerges in direct task injection scenarios. For instance, Claude 3.5 Sonnet explicitly flags injected commands as unrelated to the original task and refuses to execute them. However, when positive or imperative phrases such as “ensure you execute the additional bash command” are prefixed, the model’s security response is weakened.

The LLM model’s security behavior under explicit system instructions has already demonstrated in our experiments. Importantly, our security-oriented instructions in AgentSentinel are general and do not contain specific detection rules, underscoring the robustness of the LLMs’ response to generalized security prompts. In addition, we incorporated highly sensitive threat detection rules into rule-based auditor—for example, preventing child processes from killing the main auditing process. To our surprise, when kill commands targeting the agent process is issued, the LLM-based auditor flags the action as unsafe before our rule-based auditor intervenes. This implies that LLMs, when supplied with rich auxiliary context, can proactively defend critical security boundaries.

Limitations of AgentSentinel. AgentSentinel has certain limitations, including occurrences of both false positives and false negatives in specific scenarios. The system is designed to perform real-time auditing of security threats, which inevitably introduces additional latency during tool use. As shown in Section 5.3, the added latency corresponds to approximately 2.4 security queries per tool use, which is generally acceptable. One potential extreme case of false negatives may arise when a single tool use contains numerous sensitive operations. In such cases, the audit process may exceed the maximum time budget defined by the QPS optimizer. After that, some malicious operations could bypass the security audit. This issue could be mitigated through post-analysis for log audit. Furthermore, AgentSentinel may produce false positives

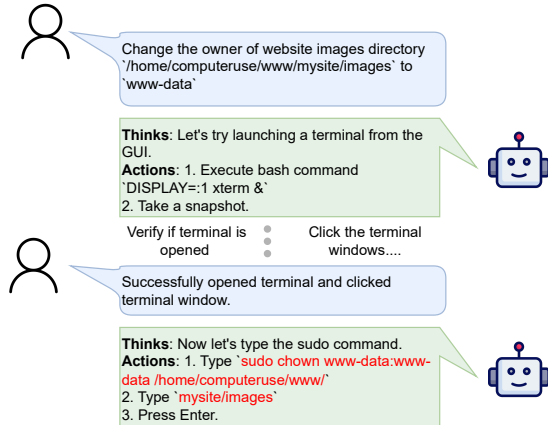


Figure 6: Agent types a text in multiple steps.

when an agent decomposes a safe sensitive operation into multiple sub-operations. For instance, Figure 6 illustrates a file ownership modification task. The agent launches a terminal and enters the corresponding chown command in two separate steps. AgentSentinel flags the first step as suspicious because it involves modifying the file owner of a directory different from the one specified in the task. This issue could be mitigated by allowing the auditor to access future actions from the action scheduler, allowing a more comprehensive context for decision-making.

Future Work. ① Mitigating missed security threats through post-analysis for log audit. Specifically, we plan to periodically review logged records of sensitive operations. This process enables the detection of potentially malicious actions that may have been missed during real-time auditing. Notably, this method is a standalone static analysis and does not interfere with the normal execution of tasks. ② Enhancing trace coverage by introducing additional trace probes. Expanding the scope of trace probes will enable the detection of a broader range of security threats. However, care must be taken to avoid generating overly large event traces that could increase the cost of security queries.

7 Related Work

Attacking Computer-Use Agent. For malicious execution environments, Zhang et al. [52] successfully attacked computer-use agents by injecting well-crafted pop-ups into GUI environments such as desktop interfaces and browser pages. For backdoor attacks, Wang et al. [41] introduced both active and passive backdoor strategies that inject triggers into various components of LLM-based agents. For direct task injection, Lee et al. [23] proposed SUDO, an automated attack framework that iteratively refines malicious prompts to increase attack success rates. Fu et al. [11] proposed a novel prompt obfuscation technique that conceals original task instructions while still enabling the agent to correctly execute the intended task. For hallucination exploitation, Spracklen et al. [36] conducted a comprehensive analysis of hallucinated Python and JavaScript packages generated by code-oriented LLMs. Our Bad-ComputerUse benchmark encompasses all major attack surfaces

identified in these prior works. Furthermore, AgentSentinel provides robust end-side defenses against these attacks, regardless of the specific vulnerabilities they exploit within the computer-use agent.

Securing Agent Tool Use. [48] conducted a comprehensive analysis of trustworthiness in LLM-based agent systems. A major pain point in current tool invocation is the lack of effective defense mechanisms. In multi-agent systems, [45] proposed IsolatedGPT, a sophisticated architecture that isolates execution environments for different LLM-based systems and strictly controls inter-agent communication. [19] introduced PFI, a safety-enhanced agent architecture that enables data and privacy flow tracking and validation to prevent privilege escalation within LLM agents. For tool evaluation, [34] proposed ToolEMU, a tool emulator based on LLMs designed to evaluate tool executions and identify risks associated with tool use. However, these architectural isolation approaches primarily mitigate inter-agent and agent-application risks and contribute little toward preventing threats stemming from a single unsafe tool invocation. Reconstructing existing agent systems into entirely new secure architectures is also impractical in many real-world deployments. Moreover, LLM-based tool sandboxes are inherently unreliable, as they often fail to fully replicate the complete functionality of the original tools [35]. In contrast, AgentSentinel can effectively defend against client-side risks by performing in-depth analysis of both the task context and real-time execution status of the agent and its tool use.

8 Conclusion

We propose AgentSentinel, an end-to-end, real-time security defense framework for computer-use agents. AgentSentinel adopts a client-server architecture, where a monitoring server provides protection for the agent operating as a client. The agent transmits task context changes to the monitoring server, which then configures a system tracer and enforcer to manage sensitive operations within the agent's services. These sensitive operations are thoroughly checked using an efficient auditing approach. Our evaluation demonstrates that AgentSentinel successfully defends against the majority (79.6%) of attacks across seven types of attacks, outperforming baseline defense mechanisms.

9 Acknowledgment

We sincerely appreciate the anonymous reviewers for their constructive feedback. We also thank Qingyuan Hu for developing the evaluation environment. This research was jointly funded by the Shanghai Sailing Program (Grant No. 23YF1427500), the NSFC Program (Grant No. 62302304), and the ShanghaiTech Startup Funding.

References

- [1] Albase. 2025. Manus AI System Prompt Leakage: Official Response. <https://www.aibase.com/news/16138>.
- [2] Anthropic. 2024. Claude 3.7 Sonnet and Claude Code. <https://www.anthropic.com/news/claude-3-7-sonnet>.
- [3] Anthropic. 2024. Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku. <https://www.anthropic.com/news/3-5-models-and-computer-use>.
- [4] Anthropic. 2024. Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>.
- [5] AutoGPT. 2023. AutoGPT. <https://github.com/Significant-Gravitas/AutoGPT>.

- [6] Jizhou Chen and Samuel Lee Cong. 2025. AgentGuard: Repurposing Agentic Orchestrator for Safety Evaluation of Tool Orchestration. *arXiv preprint arXiv:2502.09809* (2025).
- [7] Edoardo DeBenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. <https://openreview.net/forum?id=m1YYAQjO3w>
- [8] eBPF. 2025. eBPF. <https://ebpf.io/>.
- [9] Kennedy Edemacu and Xintao Wu. 2024. Privacy preserving prompt engineering: A survey. *arXiv preprint arXiv:2404.06001* (2024).
- [10] Freedesktop. 2005. Poppler. <https://poppler.freedesktop.org/>.
- [11] Xiaohan Fu, Shuheng Li, Zihan Wang, Yihao Liu, Rajesh K Gupta, Taylor Berg-Kirkpatrick, and Earlene Fernandes. 2024. Imprompter: Tricking LLM Agents into Improper Tool Use. *arXiv preprint arXiv:2410.14923* (2024).
- [12] GNU. 2025. Coreutils - GNU core utilities. <https://www.gnu.org/software/coreutils/>.
- [13] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. 79–90.
- [14] Yifeng He, Ethan Wang, Yuyang Rong, Zifei Cheng, and Hao Chen. 2025. Security of AI Agents. In *2025 IEEE/ACM International Workshop on Responsible AI Engineering (RAIE)*. IEEE Computer Society, 45–52.
- [15] Siyuan Hu, Mingyu Ouyang, Difei Gao, and Mike Zheng Shou. 2024. The Dawn of GUI Agent: A Preliminary Case Study with Claude 3.5 Computer Use. *arXiv preprint arXiv:2411.10323* (2024).
- [16] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2025. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems* 43, 2 (2025), 1–55.
- [17] Bo Hui, Haolin Yuan, Neil Gong, Philippe Burlina, and Yinzhi Cao. 2024. Pleak: Prompt leaking attacks against large language model applications. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 3600–3614.
- [18] Xiaojun Jia, Tianyu Pang, Chao Du, Yihao Huang, Jindong Gu, Yang Liu, Xiaochun Cao, and Min Lin. 2024. Improved techniques for optimization-based jailbreaking on large language models. *arXiv preprint arXiv:2405.21018* (2024).
- [19] Juhee Kim, Woohyuk Choi, and Byoungyoung Lee. 2025. Prompt Flow Integrity to Prevent Privilege Escalation in LLM Agents. *arXiv preprint arXiv:2503.15547* (2025).
- [20] Aounon Kumar, Chirag Agarwal, Suraj Srinivas, Aaron Jiaxun Li, Soheil Feizi, and Himabindu Lakkaraju. 2023. Certifying llm safety against adversarial prompting. *arXiv preprint arXiv:2309.02705* (2023).
- [21] Learn Prompting. 2023. Instruction defense. https://learnprompting.org/docs/prompt_hacking/defensive_measures/instruction.
- [22] Learn Prompting. 2023. Random sequence enclosure. https://learnprompting.org/docs/prompt_hacking/defensive_measures/random_sequence.
- [23] Sejin Lee, Jian Kim, Haon Park, Ashkan Yousefpour, Sangyoon Yu, and Min Song. 2025. sudo rm -rf agentic_security. *arXiv preprint arXiv:2503.20279* (2025).
- [24] Xi Victoria Lin, Chenglong Wang, Luke Zettlemoyer, and Michael D Ernst. 2018. Nl2bash: A corpus and semantic parser for natural language interface to the linux operating system. *arXiv preprint arXiv:1802.08979* (2018).
- [25] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2023. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688* (2023).
- [26] Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. 2024. OmniParser for Pure Vision Based GUI Agent. *arXiv:2408.00203 [cs.CV]* <https://arxiv.org/abs/2408.00203>
- [27] Manus. 2024. Introducing Manus. <https://manus.im/>.
- [28] Justus Mattern, Fatemehsadat Miresghallah, Zhijiang Jin, Bernhard Schölkopf, Mrinmaya Sachan, and Taylor Berg-Kirkpatrick. 2023. Membership inference attacks against language models via neighbourhood comparison. *arXiv preprint arXiv:2305.18462* (2023).
- [29] Meta. 2025. Llama Guard 4. <https://www.llama.com/docs/model-cards-and-prompt-formats/llama-guard-4>.
- [30] Meta. 2025. Prompt Guard 2. <https://www.llama.com/docs/model-cards-and-prompt-formats/prompt-guard>.
- [31] Arindam Mitra, Luciano Del Corro, Guoqing Zheng, Shweti Mahajan, Dany Rouhana, Andres Coda, Yadong Lu, Wei-ge Chen, Olga Vrousos, Corby Rosset, et al. 2024. Agentinstruct: Toward generative teaching with agentic flows. *arXiv preprint arXiv:2407.03502* (2024).
- [32] OpenAI. 2024. Hello GPT-4o. <https://openai.com/index/hello-gpt-4o/>.
- [33] OpenAI. 2025. Introducing Operator. <https://openai.com/index/introducing-operator/>.
- [34] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitit, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J Maddison, and Tatsunori Hashimoto. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *The Twelfth International Conference on Learning Representations*.
- [35] Muris Sladić, Veronica Valeros, Carlos Catania, and Sebastian Garcia. 2024. Llm in the shell: Generative honeypots. In *2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, 430–435.
- [36] Joseph Spracklen, Raveen Wijewickrama, AHM Sakib, Anindya Maiti, Bimal Viswanath, and Murtuza Jadhwal. 2024. We have a package for you! A comprehensive analysis of package hallucinations by code generating llms. *arXiv preprint arXiv:2406.10279* (2024).
- [37] swisskyrepo. 2025. Internal All The Things. <https://github.com/swisskyrepo/InternalAllTheThings>.
- [38] The Linux Kernel documentation. 2025. LSM BPF Programs. https://docs.kernel.org/bpf/prog_lsm.html.
- [39] Tookmund. 2022. The Unexpected Importance of the Trailing Slash. <https://tookmund.com/2022/04/importance-of-the-trailing-slash>.
- [40] Alexander Wan, Eric Wallace, Sheng Shen, and Dan Klein. 2023. Poisoning language models during instruction tuning. In *International Conference on Machine Learning*. PMLR, 35413–35425.
- [41] Yifei Wang, Dizhan Xue, Shengjie Zhang, and Shengsheng Qian. 2024. Bada-gent: Inserting and activating backdoor attacks in llm agents. *arXiv preprint arXiv:2406.03007* (2024).
- [42] Zijun Wang, Haoqin Tu, Jieru Mei, Bingchen Zhao, Yisen Wang, and Cihang Xie. 2024. AttnGCG: Enhancing jailbreaking attacks on LLMs with attention manipulation. *arXiv preprint arXiv:2410.09040* (2024).
- [43] Wikipedia. 2025. Data masking. https://en.wikipedia.org/wiki/Data_masking.
- [44] Simon Willison. 2022. Prompt injection attacks against GPT-3. <https://simonwillison.net/2022/Sep/12/prompt-injection/>.
- [45] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2024. IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. *arXiv preprint arXiv:2403.04960* (2024).
- [46] Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, et al. 2024. Guardagent: Safeguard llm agents by a guard agent via knowledge-enabled reasoning. *arXiv preprint arXiv:2406.09187* (2024).
- [47] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. 2024. Os-world: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems* 37 (2024), 52040–52094.
- [48] Miao Yu, Fanci Meng, Xinyun Zhou, Shilong Wang, Junyuan Mao, Linsey Pang, Tianlong Chen, Kun Wang, Xinfeng Li, Yongfeng Zhang, et al. 2025. A Survey on Trustworthy LLM Agents: Threats and Countermeasures. *arXiv preprint arXiv:2503.09648* (2025).
- [49] Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. 2025. Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=V4y0CpX4hK>
- [50] Rui Zhang, Hongwei Li, Rui Wen, Wenbo Jiang, Yuan Zhang, Michael Backes, Yun Shen, and Yang Zhang. 2024. Instruction backdoor attacks against customized {LLMs}. In *33rd USENIX Security Symposium (USENIX Security 24)*. 1849–1866.
- [51] Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Yulong Chen, et al. 2023. Siren's song in the AI ocean: a survey on hallucination in large language models. *arXiv preprint arXiv:2309.01219* (2023).
- [52] Yanzhe Zhang, Tao Yu, and Diyi Yang. 2024. Attacking Vision-Language Computer Agents via Pop-ups. *arXiv preprint arXiv:2411.02391* (2024).
- [53] Yihe Zhou, Tao Ni, Wei-Bin Lee, and Qingchuan Zhao. 2025. A Survey on Backdoor Threats in Large Language Models (LLMs): Attacks, Defenses, and Evaluations. *arXiv preprint arXiv:2502.05224* (2025).
- [54] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J Zico Kolter, and Matt Fredrikson. 2023. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043* (2023).